

The Engineering Playbook

Principles, practices & conventions
for building software that lasts.

Twenty-two field guides on architecture, craft,
process, and operations — the standards
we hold ourselves to on every line of code.

EDITION 2026.1
5 PARTS • 22 CHAPTERS

Code is read far more often than it is written.

This playbook is the distillation of how we build software — not as abstract theory, but as the working agreements that govern our day-to-day decisions. It exists so that any engineer, on any team, can make a choice the way the rest of us would, without needing to ask.

We believe great systems are **independent of frameworks, testable in isolation, and forgiving of change**. We believe the **main** branch should always be deployable, that small pull requests get rigorous reviews while large ones get rubber-stamped, and that **performance, accessibility, and security are features** — not afterthoughts bolted on at the end.

What follows is organized into five parts: the **foundations** of how we think about code; the **way we work** together; how we build the **backend** and the **frontend**; and how we keep it all **running in production**. Read it front to back, or jump to the guide you need. Either way, the goal is the same — to make the right thing the easy thing.

Contents

5 PARTS • 22 CHAPTERS

Part I Foundations & Philosophy

01	Clean Architecture
02	Clean Code
03	The Foundation Stack

Part II The Way of Working

04	The Way of Working
05	Git Workflow & Collaboration
06	Squash & Merge
07	Code Review
08	Feature Flags & Dark Launching
09	Developer Onboarding

Part III The Backend

10	API Design & Idempotency
11	Security Practices
12	Data Privacy & Compliance
13	Third-Party Integrations

Part IV The Frontend

14	State Management
15	Frontend Resilience & Offline
16	Styling & CSS Conventions
17	Accessibility
18	Timezones, Currency & Localization

Part V Running in Production

19	Testing Strategy
20	Performance & Optimization
21	Observability & Incident Response
22	Disaster Recovery & Backups

PART ONE

Foundations & Philosophy

Before tools and process come ideas. How we structure systems, name things, and decide what depends on what — the bedrock everything else is built on.

01 Clean Architecture

02 Clean Code

03 The Foundation Stack

Clean Architecture

Separate a system into concentric layers so that business rules never depend on the database, the framework, or the screen.

Clean Architecture, introduced by Robert C. Martin, is a design philosophy that separates the elements of a design into ring levels. The goal is a system that is independent of its details — and therefore easy to test, change, and outlive its tools.

What clean architecture buys us

- **Independent of frameworks.** The architecture doesn't depend on some feature-laden library. Frameworks become tools you use, not constraints you cram your system into.
- **Testable.** Business rules can be tested without the UI, database, or web server.
- **Independent of the UI.** A web UI could be swapped for a console UI without touching the business rules.
- **Independent of the database.** Swap PostgreSQL for MongoDB and the business rules don't notice.
- **Independent of any external agency.** The core simply knows nothing about the outside world.

THE DEPENDENCY RULE

Source code dependencies must point **only inward**, toward higher-level policies. Nothing in an inner circle can know anything about something in an outer circle — not even its name.

The layers, innermost to outermost

1 • Entities — Enterprise Business Rules

Entities encapsulate the most general, high-level business rules. They are the least likely to change when something external changes, and in a clean architecture they depend on nothing else at all.

2 • Use Cases — Application Business Rules

This layer contains application-specific rules. Use cases orchestrate the flow of data to and from the entities. Changes here shouldn't affect entities; changes to the database or UI shouldn't affect use cases.

3 • Interface Adapters — Controllers, Gateways, Presenters

Adapters convert data from the format convenient for use cases and entities into the format convenient for an external agency such as the database or the web. The entire MVC architecture of a GUI lives here. No code inward of this circle should know anything about the database — if it's SQL, all the SQL stays in this layer.

4 • Frameworks & Drivers — Web, DB, Devices

The outermost layer is frameworks and tools: the database, the web framework, devices. You write little here beyond glue code. The web is a detail. The database is a detail. We keep these things on the outside where they can do little harm.

Crossing boundaries with Dependency Inversion

If dependencies only point inward, how does a Use Case (layer 2) talk to the Database (layer 4)? It doesn't call the database directly — that would violate the rule. Instead:

- 1 The Use Case defines an **interface** (e.g. `UserRepository`) in its own layer.
- 2 The outer adapters layer provides a class (e.g. `PostgresUserRepository`) that implements it.

At compile time the Use Case depends only on its own inner interface; at runtime the database adapter is injected to fulfill it. When an external part becomes obsolete — the database, the web framework — you replace it with a minimum of fuss.

Clean Code

Code that is easy to read, understand, and maintain by anyone on the team — written with the intent that other humans will need to change it.

Clean code isn't just code that executes correctly; it's code written knowing that other people — including your future self — will read, modify, and extend it. These are the principles we hold to.

1 • Meaningful names

- **Intention-revealing.** Names should answer why something exists, what it does, and how it's used. Prefer `elapsedTimeInDays` over `d`.
- **Pronounceable and searchable.** Use words you can say out loud and find in the codebase.
- **Classes are nouns, functions are verbs.** `Customer`, `Account` — `saveCustomer`, `deletePage`.

2 • Functions should be small

- **Do one thing.** A function should do one thing, do it well, and do it only (the Single Responsibility Principle).
- **Keep them short.** If a function needs significant scrolling to read, it's doing too much.
- **Fewer arguments.** Zero, one, or two. Three should be avoided; more than three needs special justification (or an object).

3 • Comments don't make up for bad code

Always try to explain your intent in the code itself, through good names. Use comments to explain the *why*, not the *what* — to clarify obscure business logic, document legal constraints, or warn of consequences. Remove commented-out code and redundant comments that just restate the code.

4 • Formatting & structure

- **Vertical density.** Closely related concepts belong vertically close together.
- **Consistent, automated style.** Agree on one style and automate it with Prettier, `ktfmt`, or `google-java-format`.
- **The newspaper metaphor.** A source file should read like an article: high-level concepts at the top, detail increasing as you move down.

5 • Error handling

- **Use exceptions, not error codes.** Error codes clutter the caller's logic.
- **Don't return null.** Use Optionals or empty collections instead of forcing null-checks.
- **Don't pass null** into methods whenever you can avoid it.

6 • DRY — Don't Repeat Yourself

Every piece of knowledge must have a single, unambiguous, authoritative representation in the system. Duplication is the root of many maintenance evils.

7 • Clean tests

Test code is just as important as production code and must be kept clean.

Tests follow the **F.I.R.S.T.** principles — Fast, Independent, Repeatable, Self-validating, Timely — and a clear **Arrange-Act-Assert** (Given-When-Then) structure.

8 • Language-specific guidelines

React

- Keep components small and focused; break complex UI into reusable sub-components.
- Extract complex logic or side effects into custom hooks.
- Avoid prop drilling — use Context, Zustand, or composition.
- Never mutate state directly; treat state and props as immutable.

TypeScript

- Avoid `any`. Use `unknown` and narrow it, or define a proper type.
- Use `interface` for object contracts; `type` for unions and aliases.
- Replace magic strings with enums or string-literal unions: `type Status = 'open' | 'closed'`.

Kotlin

- Write idiomatic Kotlin: `data class` for models, `when` over long if-else chains, extension functions for utilities.
- Use null safety fully (`?`, `?:`); avoid the `!!` assertion.
- Default to `val` and read-only collections over `var` and mutable variants.

Java

- Use modern features: `record` for immutable data, `var` for obvious local types, the Stream API for declarative processing.
- Keep fields `private`; prefer behavior-rich domain models over anemic data structures.
- Return `Optional<T>` instead of `null`.

“The ratio of time spent reading versus writing is well over 10 to 1. Making code easy to read makes it easier to write.”

— ROBERT C. MARTIN

The Foundation Stack

The default technology choices, tooling, and deployment strategy behind every project we start.

When starting a new project, the first decision is architectural: does it need a full backend, or is a local-first approach enough?

Architectural paradigms

Local-first PWA — default for standalone apps

The app runs entirely in the browser and stores data locally, with no custom backend. Storage uses **IndexedDB** via the **Dexie.js** wrapper. Ideal for single-user tools, calculators, offline utilities, or anywhere data shouldn't leave the device.

REST API backend — for distributed / multiplayer apps

A client-server architecture where the frontend talks to a remote backend. Used for multi-user platforms, centralized processing, or integration with external webhooks and services.

Frontend stack

- **Framework:** React + TypeScript.
- **Build tool:** Vite.
- **Package manager:** PNPM (Node 24 required).
- **PWA:** `vite-plugin-pwa` with `generateSW`, plus a prompt-for-update flow when code changes.
- **State & fetching:** TanStack Query for async state; Zustand or React Context for local UI state.
- **Styling:** Tailwind CSS.

Testing: **Vitest** for unit/integration and **Playwright + Serenity BDD** for E2E, using the Screenplay Pattern with Serenity reports enabled.

Backend stack (if required)

- **Language:** Kotlin, with Hexagonal / Clean Architecture.
- **Default:** Spring Boot with JPA backed by PostgreSQL.
- **Lightweight alternative:** Ktor with Exposed on PostgreSQL.
- **BaaS alternative:** Supabase (PostgreSQL, auth, real-time APIs).

APIs are standardized with **OpenAPI**, implement **HATEOAS** links that adjust by resource state and permissions, and expose a version endpoint (e.g. via Spring Boot Actuator).

Database migrations & data modeling

Migrations are managed with **Flyway** and applied automatically on application startup, so the schema is always in sync with the code. Conventions: `snake_case` names, plural tables (`users`, `orders`), explicit UUID primary keys, and `created_at` / `updated_at` on every table.

ZERO-DOWNTIME · EXPAND & CONTRACT

Because old and new instances run side by side during a rolling deploy, **every migration must be backwards compatible**. Make breaking changes across three deployments: **Expand** (add the new column, write to both), **Migrate** (backfill, read from the new column), **Contract** (drop the old column once nothing reads it).

Git hooks & formatting

Code is auto-formatted on commit with **Husky** and **lint-staged**: ESLint & Prettier for JS/TS, `ktfmt` for Kotlin, `google-java-format` for Java.

CI/CD & versioning

- **CI**: GitHub Actions. **Dependencies**: Renovate keeps them current.
- **Lockstep versioning**: in a monorepo, frontend and backend share one global version, bumped on every merge to `main`.
- The version is visible in the app UI and returned by the backend version endpoint.

Deployment & analytics

Frontends deploy to **Cloudflare Pages** with **Cloudflare Web Analytics**. Because the analytics are privacy-first and use no client-side state, they require no GDPR cookie banner — keeping the UI clean and compliant by default.

DEVELOPER EXPERIENCE · THE RUN SCRIPT

Every repository contains a `run` shell script in its root that automates all developer commands (`./run app:build` , `./run db:up`). It acts as executable documentation — nobody has to memorize CLI arguments, and the script itself documents how to work in the environment.

PART TWO

The Way of Working

How we manage work is just as important as how we write code. Tickets, branches, reviews, flags, and the daily rituals that keep always deployable.

04 The Way of Working

05 Git Workflow & Collaboration

06 Squash & Merge

07 Code Review

08 Feature Flags & Dark Launching

09 Developer Onboarding

The Way of Working

Good tickets, a clear Definition of Done, and continuous deployment
— so work flows without friction or guesswork.

1 • The anatomy of a good ticket

A Jira or Linear ticket must contain enough context that a developer can pick it up without asking the author for clarification.

- **Bugs** include "Steps to Reproduce", "Expected Behavior", and "Actual Behavior".
- **Features** include clear "Acceptance Criteria" — *Given X, When I do Y, Then Z happens.*

2 • Definition of Done

A task is not "Done" when the code works locally. A ticket moves to Done only when:

- 1 Code is written and self-reviewed.
- 2 Unit / integration tests are written and passing in CI.
- 3 Code has been reviewed and approved by at least one peer.
- 4 The PR is merged into `main`.
- 5 If hidden behind a feature flag, the flag has been verified in production.

3 • Continuous deployment

We don't have "release days." Code merged into `main` is automatically deployed to production by the CI/CD pipeline. To support that, all work follows our Git workflow and feature-flag guidelines.

Git Workflow & Collaboration

A lightweight, trunk-based flow that keeps history clean, traceable, and always deployable.

1 • Branching strategy

We use a lightweight model — GitHub Flow / trunk-based. Long-lived feature branches are discouraged.

- **main** is the single source of truth and is always deployable.
- **Feature branches** are cut from **main** (**feature/login-page** , **fix/header-alignment**).
- **Merge strategy**: squash and merge into **main** for a clean, linear history.

2 • Commit conventions

We follow **Conventional Commits**, which makes history readable and enables automated changelogs and semantic versioning.

FORMAT

```
<type>(<optional scope>): <description>
```

```
feat(auth): add JWT session refresh logic
```

Types: **feat** , **fix** , **docs** , **style** , **refactor** , **perf** , **test** , **chore** .

3 • Pull requests & reviews

- **Keep PRs small** — reviewable in under 15 minutes, typically under 400 lines.
- **Descriptive titles & descriptions**: the title is a conventional commit; the description explains *why*, not just *what*.
- **Review culture**: be kind and constructive, ask questions instead of making demands, and approve when the code is better than it was. Perfect is the enemy of good.
- **CI checks**: every PR must pass formatting, linting, and tests before merge.

4 • Keeping **main** deployable

Because **main** must always be deployable, a large feature is rarely built in one massive PR. Break the work into small PRs that merge independently without breaking the app — reducing risk and speeding review.

EXAMPLE · BUILDING A "USER PROFILE" FEATURE

PR 1 — Database (hidden): add the `user_profiles` table and backend entity. Merges safely; the table sits empty.

PR 2 — Backend API (hidden): add controllers, services, and tests. The API exists and is tested, but nothing calls it yet.

PR 3 — Frontend (dark launched): build the components and route, but don't add the nav link (or hide it behind a flag). Developers can test by URL; users won't stumble on it.

PR 4 — The reveal (live): add the "My Profile" button to the navigation. The feature is now live.

Slicing work this way avoids "merge hell," shrinks the blast radius of bugs, and keeps `main` continuously deliverable.

Squash & Merge

Keep your messy, granular commits while you work — but land one clean, unified commit on `main`.

While working on a feature branch you make small, messy commits as you go:

FEATURE BRANCH HISTORY

```
wip: start login page  
fix: typo in css  
feat: add api call for login  
fix: actually make the api call work  
chore: cleanup comments
```

Standard merge vs. squash

With a **standard merge**, all five commits plus an extra merge commit land on `main`. Over time, across many developers, history becomes a tangled web of context-less "fix typo" commits that's nearly impossible to read.

With a **squash and merge**, Git collapses all five into **one unified commit** on `main`, named clearly per our Conventional Commits standard — e.g. `feat(auth): implement user login page`.

Why it's better

- **Clean history.** `main` reads like a high-level changelog of completed features.
- **Linear history.** No criss-crossing merge commits — just a straight line.
- **Easy reverts.** If the new login page breaks production, you revert one commit instead of untangling five scattered ones.

IN SHORT

You keep your messy, granular commits while you work on your own branch — but `main` stays pristine and professional.

Code Review

Reviews ensure architectural integrity, security, and knowledge sharing — not nitpicking.

1 • The philosophy

- **Automate the pedantry.** Formatting, linting, and imports are handled by Husky, Prettier, and ESLint / ktfmt. Don't leave comments about formatting.
- **Assume competence & good intent.** Phrase feedback as a question or suggestion, not a command.
- **Review architecture & logic.** Look for race conditions, security flaws, missing test coverage, and violations of Clean Architecture.

2 • Conventional comments

Prefix every review comment with a label so feedback is clear and actionable:

- **blocking:** cannot merge until addressed (a security flaw or major bug).
- **suggestion:** a proposed improvement the author can take or decline; does not block.
- **nit:** minor, non-blocking feedback — feel free to ignore.
- **question:** seeking clarification to understand the code.

3 • PR size & SLAs

PR SIZE

< 400

Lines of code. Large PRs get rubber-stamped; small PRs get rigorous reviews.

TURNAROUND

24h

A PR sitting for days is blocked inventory that slows the whole team.

Feature Flags & Dark Launching

Merge incomplete features into a deployable `main` by wrapping them in toggles that hide them from users.

1 • What is dark launching?

Deploying a feature to production but wrapping it in a conditional toggle — a feature flag — so it stays hidden from standard users.

2 • When to use feature flags

- When a feature takes longer to build than a single PR.
- When a risky migration needs a "kill switch" to revert instantly without a rollback.
- When rolling out to internal staff first (beta) before the public.

3 • Implementation patterns

Pattern 1 — HATEOAS-driven flags (preferred)

Because our backend is RESTful and uses HATEOAS, the most elegant approach drives flags from the API links themselves. The backend evaluates the flag and includes the action link only if the feature is enabled.

BACKEND RESPONSE

```
// if (flags.isCheckoutV2Enabled) addLink("checkout_v2")
{
  "id": "123",
  "status": "CART",
  "_links": {
    "self": { "href": "/cart/123" },
    "checkout_v2": { "href": "/cart/123/checkout/v2" }
  }
}
```

FRONTEND — CHECK FOR THE LINK

```
{cart._links.checkout_v2
  ? <CheckoutV2 url={cart._links.checkout_v2.href} />
  : <CheckoutV1 />}
```

This centralizes logic in the backend, so flag evaluation isn't duplicated across web, iOS, and Android.

Pattern 2 — Global UI toggles (fallback)

If a feature doesn't map to a REST resource (e.g. a global nav redesign), fetch global flags from a config endpoint and render conditionally.

```
{flags['new-nav-bar'] ? <NavigationBarV2 /> : <NavigationBarV1 />}
```

Pattern 3 — API guards

Always secure new endpoints. Even with the UI hidden, a malicious user could guess the URL. If the backend flag is disabled, the endpoint must return

`404 Not Found` or `403 Forbidden`.

THE FLAG LIFECYCLE • AVOIDING TECH DEBT

Feature flags are temporary technical debt. A flag must be removed within **14 days** of the feature reaching 100% of users: delete the old code path, remove the conditional, and delete the flag from configuration.

Developer Onboarding

Day one: everything you need to get a local environment running quickly.

1 • Prerequisites

- **Node.js 24+** — manage versions with `nvm` or `fnm`.
- **pnpm** — `npm install -g pnpm`.
- **Docker Desktop** (or OrbStack on Mac) for local PostgreSQL and Redis.
- **Java 21+** for backend work — manage with SDKMAN.

2 • Secrets & environment variables

We don't commit secrets to Git.

- 1 Copy the template: `cp .env.example .env`.
- 2 Ask your team lead or check the secure vault (1Password / AWS Secrets Manager) for local development keys.

3 • The `./run` script

Common tasks are automated via the `./run` script in the project root.

COMMON COMMANDS

```
./run db:up          # start local Docker containers
./run backend:start  # boot the backend server
./run frontend:start # install deps & start Vite dev server
./run help           # if you ever get stuck
```

PART THREE

The Backend

Designing APIs that are predictable and safe to retry, securing them as a first-class concern, respecting user data, and protecting the core from vendor lock-in.

10 API Design & Idempotency

11 Security Practices

12 Data Privacy & Compliance

13 Third-Party Integrations

API Design & Idempotency

Consistency is paramount: a predictable API lets frontend developers and external consumers know exactly how to interact with our system.

1 • RESTful principles

Endpoints represent resources — use nouns, not verbs.

PREFER

GET /users
POST /orders

AVOID

GET /getAllUsers
POST /createNewOrder

- **GET** — retrieve a resource or collection. Safe & idempotent.
- **POST** — create a new resource. *Not* idempotent; the server assigns the ID.
- **PUT** — fully replace a resource. Idempotent; omitted fields are set to null/empty.
- **PATCH** — partially update; send only the fields that change.
- **DELETE** — remove a resource. Idempotent; the end state is the same on retry.

2 • HATEOAS

Responses include hypermedia links that guide the client on what actions are possible next, based on resource state and user permissions.

```
{
  "id": "123",
  "status": "PENDING",
  "_links": {
    "self": { "href": "/orders/123" },
    "cancel": { "href": "/orders/123/cancel" }
  }
}
```

3 • Pagination & filtering

Never return unbounded lists. Use query parameters for filtering, sorting, and pagination, and include metadata (total pages, total items, current page).

```
GET /users?role=admin&sort=-createdAt&page=2&size=20
```

4 • Error handling

Standardize all error responses with the **RFC 7807 Problem Details** format.
Never expose internal stack traces or database exceptions to the client.

```
{
  "type": "https://api.example.com/errors/out-of-stock",
  "title": "Out of Stock",
  "status": 409,
  "detail": "Item 456 is no longer available in the requested quantity."
}
```

5 • Versioning

- Use URI versioning for major, breaking changes — `/api/v1/users` .
- Non-breaking changes (adding fields or endpoints) don't require a version bump.
- Expose a root `/version` or `/actuator/info` endpoint reporting the deployed version.

6 • Idempotency & safe retries

Frontends often auto-retry on network drops. `GET` , `PUT` , and `DELETE` are naturally idempotent, but `POST` requests — creating an order, submitting a payment — are not.

IDEMPOTENCY KEYS

The client generates a unique UUID per transaction and sends it as a header (`Idempotency-Key: <UUID>`). The backend checks the key against a fast datastore before processing: if **new**, process and cache the response; if **already processed**, return the cached response without re-running the logic; if **currently processing** (a double-click race), lock and return `409 Conflict` .

Generating the key on the frontend

If the frontend generates a new key on every retry, idempotency fails. The key must be tied to the user's *intent*, not the network request.

- **Generation:** use `crypto.randomUUID()` .
- **Automatic retries:** generate the key *before* handing the request to TanStack Query, so retries resend identical headers.
- **Double-clicks:** store the key in a `useRef` on first click, or disable the submit button immediately.
- **Surviving reloads:** persist the UUID in `sessionStorage` until success, or reuse a related entity ID (like a pre-generated `order_id`).

Security Practices

Security is a first-class citizen of the development lifecycle, not an afterthought.

1 • Secrets management

- **Never commit secrets.** Passwords, API keys, and certificates stay out of version control.
- **Environment variables.** Use `.env` locally — and make sure `.env` is in `.gitignore`.
- **Production secrets** are injected via the CI/CD pipeline or a dedicated manager (AWS Secrets Manager, HashiCorp Vault, Cloudflare Secrets).

2 • Authentication & authorization

- **Authentication (who are you?):** use OAuth2 or OIDC. Don't roll your own crypto; for passwords use Argon2 or bcrypt.
- **Authorization (what can you do?):** validate permissions on *every* protected request. The backend — not the frontend — is the final authority.
- **JWTs:** keep lifespans short and use refresh tokens for new access tokens.

3 • Protecting against the OWASP Top 10

- **SQL injection:** never concatenate strings into queries. Use parameterized queries (built into JPA/Hibernate, Exposed).
- **XSS:** React escapes JSX values by default. Avoid `dangerouslySetInnerHTML` unless the input is aggressively sanitized (e.g. DOMPurify).
- **CSRF:** with session cookies, use anti-CSRF tokens or `SameSite=Strict`.

4 • Dependency management & scanning

Use **Renovate** to keep npm, Gradle, and GitHub Actions dependencies current — staying on latest versions sharply reduces exposure to known vulnerabilities. Pair it with GitHub Dependabot or `npm audit` to alert the team to critical packages needing out-of-band patches.

PRINCIPLE OF LEAST PRIVILEGE

Services, databases, and CI/CD pipelines should hold only the minimum permissions necessary to perform their function — nothing more.

Data Privacy & Compliance

Protect users' data and comply with global frameworks like GDPR and CCPA.

1 • Data minimization

Never collect or store personal data that isn't strictly necessary for the application's core functionality. If you don't store it, it can't be breached.

2 • Hard vs. soft deletes — the Right to be Forgotten

When a user requests deletion, balance their right to be forgotten against financial and analytical integrity.

- **Soft deletion.** Don't run a SQL `DELETE` on the user record. Anonymize the PII fields (name → `Deleted User`, email → `deleted-123@example.com`) and flip an `is_deleted = true` flag.
- **Orphaned records.** This keeps linked records (past orders, payments) from breaking foreign-key constraints while purging the user's personal identity.

ENCRYPTION AT REST

All databases must be configured to use encryption at rest. If a bad actor gains access to the physical drives, the data must remain unreadable.

Third-Party Integrations

Rely on SaaS for undifferentiated heavy lifting — but protect the core architecture from vendor lock-in.

1 • The build-vs-buy decision

- **Buy** when a service solves a problem that isn't our competitive advantage — emails via SendGrid, payments via Stripe, auth via Auth0/Supabase.
- **Build** when the feature is the core value proposition of the business.

2 • The Adapter Pattern (Anti-Corruption Layer)

Never leak third-party SDK code into core business logic.

AVOID

```
// StripeClient injected
// directly into OrderService.
// Switching to PayPal means
// rewriting OrderService.
```

PREFER

```
// Domain interface:
//   PaymentProcessor
// Impl: StripePaymentProcessor
// OrderService knows only
// the interface.
```

3 • Mocking vendors

Because we use the Adapter Pattern, we can drop in a `MockPaymentProcessor` for local environments and test suites — letting developers work offline without sandbox API keys for every service.

PART FOUR

The Frontend

Managing state by category, degrading gracefully when the network fails, styling with discipline, and building interfaces everyone can use.

14 State Management

15 Frontend Resilience & Offline

16 Styling & CSS Conventions

17 Accessibility

18 Timezones, Currency & Localization

IV

State Management

Treating all state the same leads to bloated codebases. We split it into two distinct kinds: server state and client state.

1 • Server state

Data that originates from a backend API — asynchronous, shared among users, and prone to drifting out of sync with the UI.

- **Tool:** TanStack Query (React Query).
- **Why:** caching, request deduplication, background updates, pagination, and loading/error states out of the box.
- **Rule:** don't store server data in global client state like Redux or Zustand — let TanStack Query manage it.

2 • Client state

Data that exists only in the browser — is a modal open, the current theme, form input values.

Local component state

For state relevant to a single component or its immediate children, use `useState` or `useReducer`. Keep state as local as possible.

Global client state

- When many disparate components need it, use **Zustand** or **React Context**.
- **Zustand** is preferred for frequently updating state — it prevents unnecessary re-renders better than raw Context.
- **React Context** is fine for low-frequency updates like theme or auth info.

3 • Form state

Forms have unique needs — validation, touched and dirty state. Use a dedicated library like **React Hook Form**; it integrates with schemas (Zod) and manages state internally without re-rendering on every keystroke.

4 • Immutability

Always treat state as immutable — never mutate objects or arrays directly.

AVOID

```
state.items.push(newItem)
```

PREFER

```
setItems([...state.items, newItem])
```

Frontend Resilience & Offline

A failure in the network or backend should degrade gracefully — never crash the user's browser.

1 • React Error Boundaries

Never let the whole app crash because one component threw. Wrap major sections — sidebar, main content, checkout — in **Error Boundaries**. If a component fails to render, the boundary catches it and shows a localized fallback ("This widget failed to load") while the rest of the app keeps working.

2 • Handling API failures

- **Don't show raw errors.** Never display a raw 500 string. Show a friendly fallback: "We're having trouble connecting right now. Please try again."
- **Retries.** Use TanStack Query to auto-retry failed idempotent `GET` requests — e.g. three times with exponential backoff before failing.
- **Mutations.** Don't auto-retry `POST` / `PUT` unless you've implemented strong idempotency keys, to avoid double-charging or duplicate entries.

3 • Offline-first (PWA)

When building a local-first PWA:

- The service worker caches all static assets (HTML, JS, CSS) so the app boots instantly with no Wi-Fi.
- Use **IndexedDB (Dexie.js)** to store the user's data locally.
- When the network returns, run a background sync to push local changes to the cloud.

Styling & CSS Conventions

Tailwind CSS is our styling engine — used with tokens, component extraction, and built-in dark mode.

1 • Design tokens & configuration

Never use arbitrary values like `text-[#ff0000]`. Define all brand colors, typography, and breakpoints centrally in `tailwind.config.js`, and use semantic class names — `bg-primary-500`, `text-error` rather than `bg-blue-500` — so rebranding and theming stay trivial.

2 • Component extraction

Tailwind can lead to "utility class soup." If a combination of classes repeats identically more than three times (e.g. a standard button), extract it into a reusable React component — `<Button variant="primary">`.

DON'T USE @APPLY

Avoid `@apply` in custom CSS to combine Tailwind classes — it defeats the performance benefits of Tailwind's atomic CSS generation. Stick to component extraction in React.

3 • Dark mode

Use Tailwind's `dark:` modifier out of the box, and configure the `class` strategy so users can toggle dark mode manually rather than relying solely on their OS preference.

Accessibility

Accessible software is both an ethical responsibility and a legal requirement. We target WCAG 2.1 AA.

1 • Semantic HTML is king

The easiest way to make a site accessible is to use native HTML elements properly — they handle keyboard focus and screen-reader announcements automatically.

AVOID

```
<div sc-camel-on-click={submit}  
  sc-camel-class-name="btn">  
  Submit  
</div>
```

PREFER

```
<button sc-camel-on-click={submit}>  
  Submit  
</button>
```

2 • Keyboard navigation

Every interactive element must be reachable and usable without a mouse. You must be able to **Tab** through all inputs, links, and buttons, and focus states must be visually obvious — don't remove **outline** without a high-contrast alternative.

3 • ARIA attributes

When building custom components (modals, dropdowns) where native HTML falls short, use ARIA to communicate state to screen readers — **aria-expanded="true"**, **aria-hidden="true"**, **aria-label="Close dialog"**.

4 • Color & contrast

- Text must have a contrast ratio of at least **4.5:1** against its background.
- Never use color as the *only* way to convey information — an error state needs a red border *and* an icon or message.

Timezones, Currency & Localization

Handling global data incorrectly corrupts data and frustrates users. These rules are strict.

1 • The golden rule of time

STORE EVERYTHING IN UTC

All dates and times are stored in the database in **UTC**. The backend returns ISO-8601 UTC strings (`2026-06-27T17:33:34Z`); the frontend alone parses that and formats it into the user's local timezone with `Intl.DateTimeFormat` or a library like `date-fns` .

2 • Handling currency

Never use floating-point numbers for money — floating-point math produces rounding errors (`0.1 + 0.2 = 0.30000000000000004`).

- **Store as integers.** Keep monetary values in the smallest denomination — `$10.50` is stored as `1050` cents.
- **Format with `Intl.NumberFormat`** to display the integer as a localized currency string.

3 • Internationalization (i18n)

Don't hardcode user-facing strings when the app needs multiple languages. Use an i18n library like `react-i18next` and extract strings into translation files (`en.json` , `es.json`).

PART FIVE

Running in Production

Confidence to ship comes from tests, speed, visibility, and a plan for the worst day. How we verify, measure, observe, and recover.

19 Testing Strategy

20 Performance & Optimization

21 Observability & Incident Response

22 Disaster Recovery & Backups



Testing Strategy

Automated testing gives us the confidence to refactor, upgrade, and deploy frequently.

1 • The test pyramid

- **Unit tests (many).** Individual functions, classes, or components in isolation — fast to write and run.
- **Integration tests (some).** How units work together, e.g. a use case talking to a real database repository.
- **E2E tests (few).** The whole system from the user's perspective, in a real browser against a real environment.

2 • Frontend testing

- **Tools:** Vitest for unit/integration, Playwright for E2E.
- **Components:** use React Testing Library and test *behavior* (what the user sees and does), not implementation details.
- **E2E with Serenity BDD:** use the Screenplay Pattern for business-readable tests covering critical journeys like signup and checkout.

3 • Backend testing

- **Mocking:** mock external third-party APIs to keep tests deterministic.
- **Database:** don't mock the database. Use **Testcontainers** to spin up a real PostgreSQL via Docker, so your SQL and migrations actually work.

4 • Test-Driven Development

TDD isn't mandated for every line, but it's highly encouraged for complex business logic: **Red** (write a failing test), **Green** (write the simplest code to pass), **Refactor** (clean up while the test keeps you safe).

TEST QUALITY • NO FLAKY TESTS

Tests follow Arrange-Act-Assert (Given-When-Then) and must be deterministic. Flaky tests that pass or fail randomly without code changes must be fixed or deleted immediately — they erode trust in the entire CI pipeline.

Performance & Optimization

Performance is a feature. Fast applications mean better retention, higher conversion, and better SEO.

1 • Frontend performance

- **Core Web Vitals.** Monitor LCP, INP, and CLS — Cloudflare Web Analytics can help.
- **Code splitting & lazy loading.** Don't ship one massive bundle; use React's `lazy()` and `Suspense` to load route components on demand.
- **Image optimization.** Never serve raw images. Use WebP or AVIF, and set explicit `width` / `height` to prevent layout shift.
- **Bundle size.** Audit regularly; swap heavy dependencies (Moment.js) for lighter ones (date-fns or native Intl).

2 • Backend performance

- **Database indexing.** Index columns used in `WHERE`, `JOIN`, or `ORDER BY`. A missing index is the most common cause of slow APIs.
- **The N+1 query problem.** With ORMs, fetching a list then looping to fetch relations triggers hundreds of queries. Use `JOIN FETCH` or batch fetching.
- **Caching.** Cache heavy, infrequently changing results, and use `Cache-Control` headers so Cloudflare and browsers can cache at the edge.

3 • Network optimization

Keep payloads small and paginate lists. Enable gzip or Brotli compression — handled automatically by Cloudflare Pages.

Observability & Incident Response

Understanding the internal state of a system simply by looking at its external outputs — so we can identify, debug, and fix without guessing.

1 • The three pillars

- **Logs** — discrete events that occurred ("User logged in").
- **Metrics** — aggregated data over time ("100 logins per minute").
- **Traces** — the journey of a single request across boundaries, browser to API to database and back.

2 • Structured logging

In production, all logs are output as **JSON**. This avoids slow, error-prone RegEx searches and lets aggregators (Datadog, ELK, CloudWatch) index the data as searchable columns. Pass data variables separately from the human-readable message rather than concatenating them into a sentence.

```
NODE.JS / TYPESCRIPT — PINO
```

```
logger.info({ userId: "123", itemId: "456" }, "User checkout complete");
```

```
KOTLIN / SPRING BOOT — SLF4J FLUENT API
```

```
logger.atInfo()
    .addKeyValue("userId", "123")
    .addKeyValue("itemId", "456")
    .log("User checkout complete")
```

Now you can run a fast query like `userId:123 AND level:INFO`, or chart the top `itemId`’s sold.

Use log levels properly

- **ERROR** — unexpected system failures needing developer intervention (DB connection dropped). *Not* for client mistakes like a `400` or a bad password.
- **WARN** — anomalies worth investigating if they spike (rate limits hit, deprecated API usage).
- **INFO** — significant successful business events ("User created", "Order processed").
- **DEBUG** / **TRACE** — verbose technical details; turned *off* in production.

3 • Distributed tracing & correlation IDs

Every incoming request is assigned a unique `trace_id` at the entry point, injected into the logging context (e.g. via MDC) so *every* subsequent log automatically includes it. When calling other microservices, the `trace_id` is passed along in the HTTP headers.

4 • Metrics & alerting

ALERT ON SYMPTOMS, NOT CAUSES

Alerts wake people up, so they must be actionable and tied to user experience.

Alert on symptoms the user feels — "checkout error rate > 5%", "API latency > 1000ms" — not root causes that don't affect them. CPU at

90% with healthy response times is not an emergency.

The metrics pipeline runs in three tiers:

- **Micrometer** — the vendor-neutral instrumentation facade (metrics' answer to SLF4J). Inject a `MeterRegistry` to count events, time durations, and gauge states; it exposes data at `/actuator/prometheus`.
- **Prometheus** — a time-series database that *pulls* (scrapes) the endpoint every 15–30s. Pulling means the app never wastes resources pushing, and survives Prometheus going down.
- **Grafana** — connects to Prometheus, draws live graphs with PromQL, and fires webhooks to Slack / PagerDuty / email when a query crosses a threshold.

5 • Protecting PII in logs

Logs are often shipped to third-party tools, so we never log PII or secrets.

Because PII sneaks in accidentally, follow strict practices:

- **Never log whole objects.** Avoid `logger.info("Created user", user)` — it serializes the entire entity. Pick non-sensitive fields explicitly.
- **Log IDs, not names.** Log database UUIDs rather than names or emails; support can cross-reference the UUID in the secure production database.
- **Global log scrubbers.** Configure the root logger to redact sensitive keys before printing.

NODE.JS — PINO REDACTION

```
const logger = pino({
  redact: ['user.password', 'user.email', 'ssn']
})
```

Disaster Recovery & Backups

Hope is not a strategy. We plan for catastrophic failures — a regional outage, a dropped production table, a ransomware attack.

1 • Key objectives

RPO • RECOVERY POINT

5 min

The maximum amount of data we are willing to lose.

RTO • RECOVERY TIME

1 hr

How fast the system must be restored and functional.

2 • Database backups

- **Automated snapshots.** The provider (RDS, Supabase) takes full automated snapshots daily.
- **Point-in-Time Recovery.** PITR is enabled, letting us roll the database back to any exact second within the last 7 days — our primary defense against an accidental `DROP TABLE` or a corrupting migration.

3 • Infrastructure as Code

To meet our RTO, all infrastructure — servers, load balancers, DNS — is defined in code with Terraform or Pulumi. In a total regional failure, we can deploy the entire stack to a new region via a single CI/CD script.

END OF PLAYBOOK

Make the right thing the easy thing.

Twenty-two guides, one shared standard. This playbook is a living document — it evolves as our tools, our scale, and our craft do. When in doubt, optimize for the engineer who reads this next.

THE ENGINEERING PLAYBOOK VAMONOS SOFTWARE EDITION 2026.1